Singly Linked List –
Traverse List Inorder –
Space Complexity and Algorithm

Program :

```
void TraverseListInorder(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    else
    {
        Node *current = *head;
        while (current != nullptr)
        {
            cout << current->data << " ";
            current = current->next;
        }
        cout << endl;
    }
}
….
case 22:
        {
            cout << endl;
            cout << endl;
            cout << "Traverse List In Order 1:(While Loop)" << endl;
            TraverseListInorder(&head);
            cout << endl;
            cout << endl;
            break;
        }
```

Space Complexity Analysis

1. *Input Space Complexity*

A. Parameter – Only Definition

The input space complexity refers to the space required for the input parameters of the function.

In this case, the function takes:

- ***A pointer to a pointer (Node ** head), which requires a constant amount of space, O(1).***

Thus, the total input space complexity is: O(1).

• Node **head – single pointer-to-pointer (one machine word)

Just one fixed-size parameter ⇒ constant memory.

This view only considers the memory for the parameters passed to the function. The single parameter `Node **head` is a pointer-to-a-pointer, which has a **fixed, constant size**. Therefore, the input space complexity is **O (1)**.

B. Full – data (inclusive) Definition:

Add everything reachable from the input pointer when the call begins. *head may reference a singly linked list containing n nodes, each of size sizeof(Node). Those n nodes dominate ⇒ total input footprint Θ(n).

This more common definition includes the entire data structure. If the linked list has **`n` nodes**, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

2. *Auxiliary Space Complexity*

Auxiliary space complexity refers to the extra space used by the algorithm that is not part of the input. In the case of the TraverseListInorder function, the auxiliary space is determined by the variables declared within the function:

- $\boldsymbol{The\ function\ uses\ a\ pointer\ Node\ *current\ to\ traverse\ the\ linked\ list, which\ requires\ O(1) space}$.

$\boldsymbol{Since\ the\ variable\ and\ process\ occupy\ a\ constant\ amount\ of\ space\ regardless\ of\ the\ size\ of\ the\ linked\ list, the\ auxiliary\ space\ complexity\ is}$: $\boldsymbol{O(1)}$.

What the routine itself allocates beyond its input:

• Node *current – one pointer

• A few compiler-generated temporaries / return address

• No dynamic heap allocations, no recursion, no additional data structures.

The amount of additional memory is constant, independent of n.

Auxiliary space is the extra memory an algorithm uses, not counting the input. For this function:

1. **`Node *current`**: The function declares only **one** extra variable, a pointer named `current`, to traverse the list. This pointer occupies a constant amount of memory.
2. **Iteration**: The `while` loop reuses this single `current` pointer to move through the list. No new memory is allocated during the traversal.

Since the function only needs one extra pointer to do its job, regardless of the list's length, the amount of extra memory used is constant. The space used by the output stream (`cout`) is generally not counted as part of an algorithm's auxiliary space.

Therefore, the auxiliary space complexity is **Θ (1)**.

$Algorithm$

$TraverseListInorder(START)$

1. $[START\ is\ NULL?]\ if\ START = NULL, then$:

$Write$: `` $List\ is\ Empty$. ``

$Return\ and\ Exit$

$Else$

$SET\ CURRPTR := START$ $[CURRPTR\ will\ point\ to\ Starting\ NODE]$

$Repeat\ while\ \ CURRPTR \neq NULL$:

$Write$: $INFO[CURRPTR]$

$SET\ \ CURRPTR := LINK[CURRPTR]$

$[END\ of\ LOOP]$

$[END\ of\ IF\ structure]$

$[END\ of\ Algorithm]$